

PACE Solver Description: Hitting Pair DBS and AF-DFS

Rens Versnel  

Utrecht University, The Netherlands

Abstract

This paper describes the 6th place submission to the exact track of the PACE challenge 2026. This solver consists of two parts: a depth-bounded search akin to existing FPT algorithms [2] for this problem, and a depth-first search procedure that enumerates all agreement forests of the instance using incompatible triple and path pair constraints.

2012 ACM Subject Classification Theory of computation → Parameterized complexity and exact algorithms; Mathematics of computing → Trees; Theory of computation → Dynamic programming

Keywords and phrases Maximum agreement forest, PACE Challenge, parameterized complexity, phylogenetic trees

Category PACE Solver Description

Related Version This paper is based on my master’s thesis:

Full Version: <https://studenttheses.uu.nl/handle/20.500.12932/52103>

1 Short algorithm description

The main algorithm starts by exhaustively applying common cherry reduction, single-vertex tree reduction, reduction rule 2.2.1 (from [2]), reduction rule 1 (from [2]), subtree cluster reduction and LSI cluster reduction (see Section 2). Then on each reduced cluster it applies whichever of the two algorithms (AF-DFS and HP-DBS) it considers better suited to solving it, often running AF-DFS for a short period before defaulting to HP-DBS.

The AF-DFS algorithm enumerates the agreement forests of an instance, by starting at the trivial solution (only single-vertex trees) and then traversing the set of agreement forests while incrementally checking the candidate additions against precomputed constraints. This makes AF-DFS particularly fast on instances with few agreement forests.

The HP-DBS algorithm is closer to traditional FPT algorithms, except that it stores a set of *hitting pairs* as constraints, meaning that for each pair one of the two labels must become a single-vertex tree. This set of hitting pairs leads to several reduction rules and helps choose an efficient branching rule to apply. We will describe the two algorithms separately:

1.1 Agreement forest depth-first search (AF-DFS)

There are instances of MAF where the optimal solution has an *order* (that is, a number of components) close to the number of leaves; these instances are hard to solve with depth-bounded search algorithms, but should often be relatively easy because there are very few agreement forests that need to be considered. To cover these cases, we introduce a new algorithm that essentially enumerates all possible agreement forests in a depth-first search manner, starting with the trivial solution consisting of only single-vertex trees.

The first step is to define a canonical representation of each agreement forest. Then, we define two operations on the canonical representations that are sufficient to enumerate all possible agreement forests:

■ **Algorithm 1** An exact algorithm for computing a MAF

Procedure *ITER-AF-DFS*(Ω, A)
Input: A set Ω of forests and an agreement forest A of instance Ω .
Output: A set of agreement forests of Ω reachable from A by any combination of *ExtendLastComp* and *AddNewComp* operations.
 $\mathcal{A} \leftarrow \text{ExtendLastComp}(A) \cup \text{AddNewComp}(A)$
return $\{A\} \cup \bigcup_{A' \in \mathcal{A}} \text{ITER-AF-DFS}(\Omega, A')$

Procedure *AF-DFS*(Ω)
Input: A set $\Omega = \{F_1, \dots, F_t\}$ of forests.
Output: A minimum order agreement forest for Ω .
 $A \leftarrow$ the trivial agreement forest of Ω , consisting of single-vertex trees
 $\mathcal{A} \leftarrow \text{ITER-AF-DFS}(\Omega, A)$
return the agreement forest in $\mathcal{A}$ with minimum order

41 ► **Definition 1** (Canonical representation). *Let A be an agreement forest of an instance Ω . For each component of A with at least 2 labels, list its labels in increasing order. Then order these lists in increasing order by their first (smallest) label. The resulting ordered collection $S_1, S_2, \dots, S_m$ is the canonical representation of A . Note that we may refer to an agreement forest and its canonical representation interchangeably.*

46 ► **Definition 2** (Extend last component (*ExtendLastComp*)). *This procedure takes the canonical representation of an agreement forest and returns all agreement forests that can be created from it by the addition of a previously unused label to the end of the last component (while respecting the ordering).*

50 ► **Definition 3** (Add new component (*AddNewComp*)). *This procedure takes the non-empty canonical representation of an agreement forest and returns all agreement forests that can be created from it by the addition of a new component at the end consisting of two previously unused labels (while respecting the ordering).*

54 It is clear that *any* agreement forest of an instance is reachable by a combination of *ExtendLastComp* and *AddNewComp* operations.

56 Now we are ready to formulate the depth-first search routine that enumerates all agreement forests, see Figure 1; its correctness follows from the preceding observations.

58 Determining whether a forest is an agreement forest

59 To complete the description, we will explain how we determine whether a forest or its canonical
60 representation is an agreement forest for an instance. We implemented two approaches: the
61 first approach is based on an ILP formulation for the rSPR distance problem by Wu [3].
62 Wu defines two types of constraints, incompatible triple constraints and incompatible path
63 constraints, which we modified to recognize an agreement forest for multiple binary trees
64 efficiently when its components are small. The second approach we used is to compute a
65 DP-table containing all valid components, where the i th layer contains all legal components
66 with i labels; and for each component, storing the set of labels that can be appended to it to
67 form a new valid component of size $i + 1$. Using this, we can reduce the number of invalid
68 extensions attempted in *ExtendLastComp* operations. We used this second approach as a
69 supplement to the first.

1.2 Hitting pair depth-bounded search (HP-DBS)

This depth-bounded search algorithm is similar to the FPT algorithm of Shi et al. [2], but we extend it to include hitting-pair constraints:

► **Definition 4 (Hitting Pair).** *Let X be a set of labels. A hitting pair set H is a set of unordered pairs of (two different) labels.*

The idea of a hitting pair $\{a, b\}$ is that it forces at least one of a and b to be a single-vertex tree in the current execution branch. This is a hard constraint in the recursion, making several more reduction rules possible. We will denote by $\text{HP-MAF}(\Omega, H, k)$ the modified MAF problem in which the hitting pairs in H must also be *satisfied*.

The common cherry reduction and single-vertex tree reduction rules are applied exhaustively between every other branching or reduction rule. The common cherry reduction is modified to adhere to the hitting-pair constraints by merging only pairs of labels that are not in a hitting pair together. We denote these reduction rules by HP-RR1 and HP-RR2 and we say an instance is RR1- and RR2-reduced if HP-RR1 and HP-RR2 are not applicable.

The introduction of hitting pairs also allows us to formulate a modified version of the base branching rule 2.1 as defined in [2]. Let (Ω, H, k) be a RR1- and RR2-reduced instance and let $\{a, b\}$ be a cherry in some forest in Ω with $\{a, b\} \notin H$.

► **Definition 5 (HP-Branching Rule 1 (HP-BR1)).** *Branch in two ways:*

- *cut all subtrees pendant to the path between a and b in all forests;*
- *Add $\{a, b\}$ to H .*

The safety of this branching rule follows from the proof of branching rule 2.1 in [2].

The previous branching rule is not applicable when every cherry in the instance is already a hitting pair. To complement this we introduce another branching rule that uses these hitting pairs, for which the safety should be clear:

Let (Ω, H, k) be a RR1- and RR2-reduced instance and let a be a label that is in at least one hitting pair and let B be the set of labels that form a hitting pair with a (i.e., $B = \{b \mid \{a, b\} \in H\}$).

► **Definition 6 (HP-Branching Rule 2 (HP-BR2)).** *Branch in two ways:*

- *Make a a single-vertex tree in all forests;*
- *Make all labels in B single-vertex trees in all forests.*

Despite the introduction of hitting pairs, reduction rule 2.2.1 from [2], which can have a great impact in some cases, is still applicable in most cases with some modifications:

► **Definition 7 (HP-Reduction Rule 3 (HP-RR3)).** *Let (Ω, H, k) be a RR1- and RR2-reduced instance and a and b be two labels that form a cherry in some forest and are not in any hitting pairs in H (i.e., $\nexists c$, s.t. $\{a, c\} \in H \vee \{b, c\} \in H$). If, in every forest:*

- *a and b are either a cherry or have exactly one pendant subtree on the path between them that prevents them from being a cherry;*
 - *and the label-sets of this pendant subtree are the same across all forests,*
- then cut the pendant subtree between a and b (if applicable) in each forest, thereby making a and b a cherry in each forest.*

The proof of this reduction rule is also a slightly modified version of the proof given in Lemma 4.4 in [2], which we omit due to the page limit. However, it should not be difficult to verify that their proof holds for HP-MAF when neither a nor b is in a hitting pair.

■ **Algorithm 2** HP-DBS: an exact algorithm for HP-MAF(Ω, H, k)

Input: A set Ω of X -forests and a set H of hitting pairs and a parameter k .
Output: An agreement forest for Ω of order at most k , such that every hitting pair in H is satisfied, or report that no such agreement forest exists.
if the order of Ω exceeds k **then** return NO;
if Ω contains no cherries **then** unmerge all previously merged labels and return F_1 ;
if a reduction rule from HP-RR1, HP-RR2, ..., HP-RR5 is applicable **then** apply the first applicable rule from HP-RR1, HP-RR2, ..., HP-RR5 and recurse;
if either form of cluster reduction is applicable **then** apply it and solve the clusters; apply an applicable branching rule from HP-BR1 and HP-BR2 and recursively call HP-DBS on each produced branch, returning any agreement forest found;

113 To generate more hitting pairs, two new reduction rules are introduced. The first is
 114 essentially the same as a well-known branching rule that: if labels a and b form a cherry in
 115 one forest and are in different components in another forest, then (at least) one of them must
 116 become a single-vertex tree. The safety of this rule follows from this observation.

117 ► **Definition 8** (HP-Reduction Rule 4 (HP-RR4)). *Let (Ω, H, k) be a RR1- and RR2-reduced*
 118 *instance. If two labels a and b form a cherry in some forest and are in different components*
 119 *in another forest, then add hitting pair $\{a, b\}$ to H .*

120 ► **Definition 9** (HP-Reduction Rule 5 (HP-RR5)). *Let (Ω, H, k) be a RR1- and RR2-reduced*
 121 *instance. Let C be the label-set of some component of a forest in Ω with $|C| \geq 2$. If a is*
 122 *a label of C and for every label $c \in C \setminus \{a\}$, $\{a, c\}$ is a hitting pair in H , then make a an*
 123 *single-vertex trees in each forest in Ω .*

124 The safety of the second reduction rule follows immediately: if a is not a single-vertex
 125 tree, then all others in C must be, which leaves only label a the component C .

126 Finally, the HP-DBS algorithm for solving HP-MAF is described in Figure 2. It follows
 127 that HP-DBS is also an exact algorithm for MAF.

2 Cluster reduction

129 We use two types of cluster reduction in this solver. We call the first *LSI cluster reduction*.
 130 It is applicable when each forest in an instance contains a component with the same label-set;
 131 in this case we simply extract this cluster and solve the resulting problem separately. We
 132 call the second type *subtree cluster reduction*. It is applicable when each forest has a subtree
 133 with the same label-set; in this case we can cut the subtree and solve it separately, but we
 134 need to add dummy labels to determine whether a maximum agreement forest can bridge
 135 the edge that was just cut. Kelk provides a write-up [1] on this including proof sketches.

References

- 136 — **References** —
- 137 1 Steven Kelk. A pedagogical implementation of the cluster reduction, 2026. URL: <https://skelk.sdf-eu.org/cluster/> [cited 2026-09-18].
- 138 2 Feng Shi, Jianer Chen, Qilong Feng, and Jianxin Wang. A parameterized algorithm for
 139 the maximum agreement forest problem on multiple rooted multifurcating trees. *Journal of*
 140 *Computer and System Sciences*, 97:28–44, 2018. doi:10.1016/j.jcss.2018.03.002.
- 141 3 Yufeng Wu. A practical method for exact computation of subtree prune and regraft distance.
 142 *Bioinformatics*, 25(2):190–196, 2009. doi:10.1093/bioinformatics/btn606.
- 143